

TUTORIEL

Docker sous WSL2

Installation, Images, Conteneurs & Docker Compose

Composant	Version / Détail
OS hôte	Windows 10 (21H2+) ou Windows 11
WSL2	Windows Subsystem for Linux version 2
Distribution Linux	Ubuntu 22.04 LTS (recommandé)
Docker Engine	27.x (dernière version)
Docker Compose	v2.x (intégré à Docker)

1. Concepts fondamentaux

Docker est une plateforme de **conteneurisation**. Contrairement aux machines virtuelles, les conteneurs partagent le noyau de l'OS hôte, ce qui les rend légers et rapides à démarrer.

Concept	Définition
Image	Modèle en lecture seule (comme un template de VM). Ex : ubuntu, nginx, mysql
Conteneur	Instance en cours d'exécution d'une image. On peut en lancer plusieurs depuis la même image
Dockerfile	Fichier texte décrivant comment construire une image personnalisée
Docker Compose	Outil pour orchestrer plusieurs conteneurs via un fichier YAML
Volume	Stockage persistant indépendant du cycle de vie du conteneur
Réseau	Réseau virtuel permettant la communication entre conteneurs

2. Installation de WSL2

Ouvrez PowerShell en **administrateur** :

```
# Activer WSL2 et installer Ubuntu en une seule commande
wsl --install

# Si WSL est déjà installé, s'assurer d'utiliser la version 2
wsl --set-default-version 2

# Installer Ubuntu 22.04 (si pas installé par défaut)
wsl --install -d Ubuntu-22.04

# Vérifier les distributions installées
wsl --list --verbose
```

■■ IMPORTANT

Un redémarrage de Windows est nécessaire après la première installation de WSL2. La virtualisation (VT-x/AMD-V) doit être activée dans le BIOS.

Après redémarrage, Ubuntu se lance et demande de créer un compte utilisateur Linux.

3. Installation de Docker Engine dans WSL2

Ouvrez votre terminal Ubuntu (WSL2) et exécutez :

```
# Mise à jour du système
sudo apt update && sudo apt upgrade -y

# Installer les dépendances pour ajouter le dépôt Docker
sudo apt install ca-certificates curl gnupg lsb-release -y

# Ajouter la clé GPG officielle de Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Ajouter le dépôt Docker
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Installer Docker
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-c
ompose-plugin -y

# Ajouter votre utilisateur au groupe docker (évite sudo)
sudo usermod -aG docker $USER
newgrp docker

# Démarrer le service Docker
sudo service docker start

# Vérification
docker --version
docker compose version
```

4. Commandes Docker essentielles

Manipulation des images

```
# Télécharger une image depuis Docker Hub
docker pull nginx
docker pull ubuntu:22.04

# Lister les images locales
docker images

# Supprimer une image
docker rmi nginx
```

Manipulation des conteneurs

```
# Lancer un conteneur nginx en arrière-plan
# -d = détaché (background), -p = mapping de port, --name = nom du conteneur
docker run -d -p 8080:80 --name mon-nginx nginx

# Tester : ouvrir http://localhost:8080 dans le navigateur

# Lister les conteneurs en cours d'exécution
docker ps

# Lister tous les conteneurs (y compris arrêtés)
docker ps -a

# Stopper un conteneur
docker stop mon-nginx

# Démarrer un conteneur arrêté
docker start mon-nginx

# Supprimer un conteneur
docker rm mon-nginx

# Entrer dans un conteneur en cours d'exécution
docker exec -it mon-nginx /bin/bash

# Afficher les logs d'un conteneur
docker logs mon-nginx
docker logs -f mon-nginx # En temps réel
```

5. Créer une image personnalisée (Dockerfile)

```
# Créer un dossier de projet
mkdir mon-site && cd mon-site
```

```
# Créer une page web simple
echo '<h1>Mon site Docker - Ilyass BEN EL MEKKI</h1>' > index.html

# Créer le Dockerfile
nano Dockerfile
```

Contenu du Dockerfile :

```
# Image de base officielle Nginx
FROM nginx:latest

# Copier la page HTML dans le dossier web de nginx
COPY index.html /usr/share/nginx/html/index.html

# Exposer le port 80
EXPOSE 80

# Construire l'image
docker build -t mon-site:v1 .

# Vérifier l'image créée
docker images

# Lancer le conteneur
docker run -d -p 8888:80 --name mon-site-web mon-site:v1

# Tester dans le navigateur : http://localhost:8888
```

6. Docker Compose – Exemple multi-conteneurs

Docker Compose permet de définir et de lancer plusieurs conteneurs avec un seul fichier YAML. Exemple : WordPress + MariaDB.

```
mkdir wordpress-compose && cd wordpress-compose
nano docker-compose.yml

services:
  db:
    image: mariadb:latest
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppassword
```

```
volumes:
  - db_data:/var/lib/mysql

wordpress:
  image: wordpress:latest
  restart: always
  ports:
    - '8080:80'
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wppassword
    WORDPRESS_DB_NAME: wordpress
  depends_on:
    - db

volumes:
  db_data:

# Lancer l'ensemble (en arrière-plan)
docker compose up -d

# Vérifier que les conteneurs tournent
docker compose ps

# Accéder à WordPress : http://localhost:8080

# Arrêter sans supprimer les données
docker compose stop

# Supprimer tout (conteneurs + réseau, MAIS pas les volumes)
docker compose down
```

■ VOLUMES ET PERSISTANCE

Les volumes (db_data) permettent de conserver les données même après docker compose down. Pour supprimer aussi les volumes : docker compose down -v

7. Récapitulatif des commandes clés

Commande	Rôle
docker pull IMAGE	Télécharger une image

<code>docker run -d -p HOST:CONT IMAGE</code>	Lancer un conteneur en arrière-plan avec mapping de port
<code>docker ps / docker ps -a</code>	Lister les conteneurs actifs / tous
<code>docker stop / start / rm NOM</code>	Arrêter / démarrer / supprimer un conteneur
<code>docker exec -it NOM bash</code>	Entrer dans un conteneur
<code>docker logs -f NOM</code>	Suivre les logs en temps réel
<code>docker build -t NOM:TAG .</code>	Construire une image depuis un Dockerfile
<code>docker images</code>	Lister les images locales
<code>docker compose up -d</code>	Lancer la stack Compose en arrière-plan
<code>docker compose down</code>	Arrêter et supprimer la stack Compose